Implementação 5

1. Objetivo

O objetivo da tarefa era implementar um sistema de sincronização robusto para uma estação de trem para coordenar de forma segura o embarque em múltiplas threads de passageiros nos vagões, evitando condições de corrida, deadlocks e busy-waiting, garantindo o controle de concorrência por meio de um único mutex e variáveis de condição. Para isso, foi solicitado a criação de 4 funções separadas que juntas formam o sistema funcional completo, as quais consegui desenvolver no projeto.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
station_init	OK	Comentei todas as partes posteriores da main do runner e rodei e como não deu erro o init funcionou
station_load_car	OK	Comentei todas as partes posteriores da main do runner e rodei e ele não apontou erro do load car
station_wait_for_car	OK	Rodei o `./sync_car` completo e não tive erros apontados ao wait for car.
station_on_board	OK	Rodei o `./sync_car` completo e não tive erros apontados ao station on board.

OBS: Ao compilar o normalmente com make e rodar o `./sync_car`, todas as funções serão usadas e testadas. Como a main já veio pronta foi meio difícil conseguir testar cada parte da implementação individualmente à medida que fui desenvolvendo.

3. Como Reproduzir

Compilar:

- Rode para compilar: `make`
- Rode para limpar: `make clean`

Executar:

- Rodar: `./sync_car`

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car-runner.c → Contém a estrutura principal da main do projeto e é responsável pela lógica central**

2. **sync_car.c → Contém a lógica das funções usadas no programa**

3. **sync_car.h → Onde estão definidas as funções e a struct utilizada no projeto**

Decisões técnicas:

1. **Uso exclusivo de um único pthread_mutex para todo o estado da estação → Restrição arquitetural exigida para garantir o gerenciamento unificado das contagens de assentos e passageiros → Necessidade de mapear cuidadosamente todas as rotas de execução para garantir que o lock fosse trancado e destrancado estritamente no início e fim de cada função, prevenindo deadlocks**

2. **Substituição de espera ativa por pthread_cond_wait encapsulado em laços while → Evitar o desperdício de ciclos de CPU iterando vazios e proteger o código contra spurious wakeups → As threads de passageiros e do vagão dormem e acordam de forma eficiente respondendo aos sinais pthread_cond_signal e pthread_cond_broadcast, mantendo alta performance**

3. **Inserção de uma barreira de proteção matemática na chegada de vagões → O projeto estava vulnerável à corrupção de estado caso capacidades nulas ou negativas fossem recebidas → Implementação de um early return validando se a capacidade é <= 0 logo após a obtenção do lock, ignorando trens inválidos sem quebrar a consistência das variáveis internas**

4. **Refatoração da condição do laço de espera em station_load_car → O log de erros do terminal evidenciou que a função station_load_car estava retornando cedo demais, pois a verificação inicial validava apenas os assentos e ignorava se ainda havia passageiros no meio do embarque → Adição da variável de estado passageiros_embarcando à lógica do while, obrigando a thread do trem a aguardar ativamente o término da transição de estado de cada passageiro antes de fechar as portas**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Pesquisa + Tentativa e Erro	Pesquisando como fazer as coisas que não entendo em documentações e em sites/fóruns como o Stack Overflow, e construindo o código por tentativa e erro	-

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese / Causa (se falhou)	Quando	Evidência
1	S1	Implementação do Init	Sucesso	-	20:12 - 31/10	Log + Github
2	S1	Implementação do load car	Sucesso Parcial	Ele não apontou nenhum erro de início, mas, depois que implementei outras coisas percebi que tinha um erro porque faltou mutex_lock e mutex_unlock	18:45 - 01/11	Log + Github + Print
3	S1	Implementação do wait for car	Sucesso	-	19:25 - 01/11	Log + Github
4	S1	Ajuste do load car (1)	Falha	Acho que configurei algum dos lock ou unlock de forma errada porque ele retornou com a mensagem “station_load_car() did not return immediately when no waiting passengers”	19:34 - 01/11	Log + Github + Print
5	S1	Ajuste do load car (2)	Falha	Tentei mudar os locks e unlocks, mas deu o mesmo erro	19:57 - 01/11	Log + Github

6	S1	Implementação do station on board / Ajuste do load car (3)	Falha	Tentei implementar o station on board para ver se isso resolveria o problema do load car, mas apenas mudou o erro para “Error: station_load_car returned early!”	17:46 - 02/11	Log + Github + Print
7	S1	Ajuste do load car (4)	Sucesso	-	18:23 - 02/11	Log + Github
8	S1	Testes finais	Sucesso	-	18:54 - 02/11	Log + Github + Prints

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

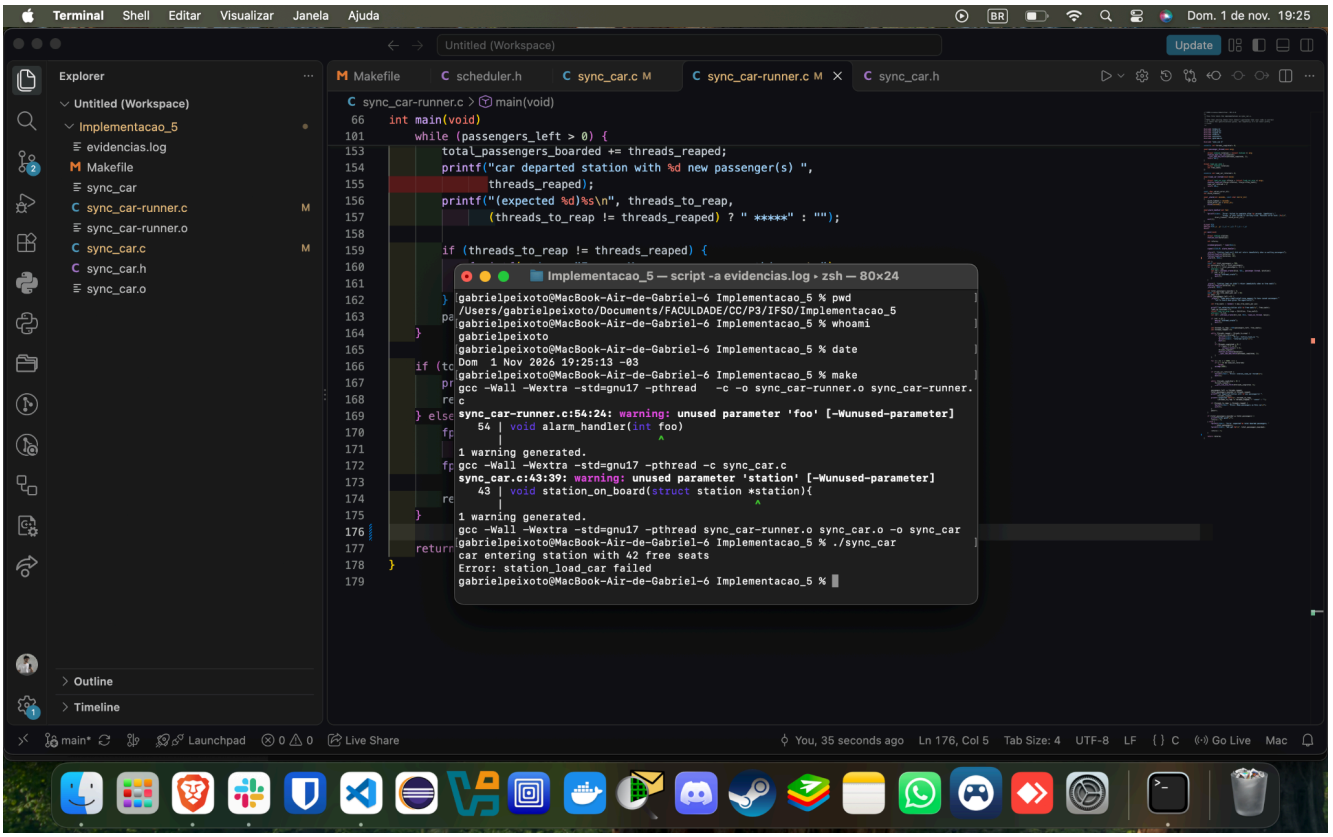
No início da sessão de trabalho, rode:

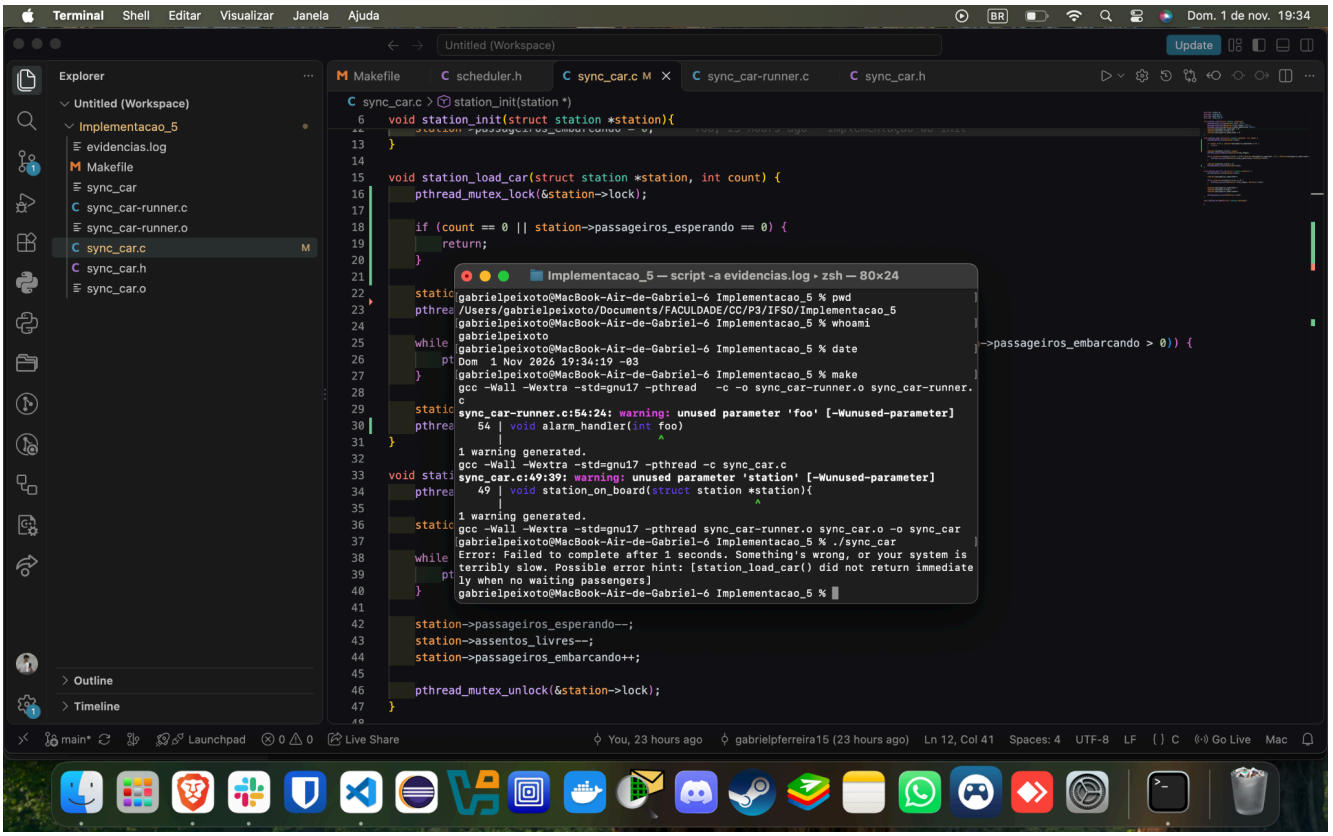
```
script -a evidencias.log
```

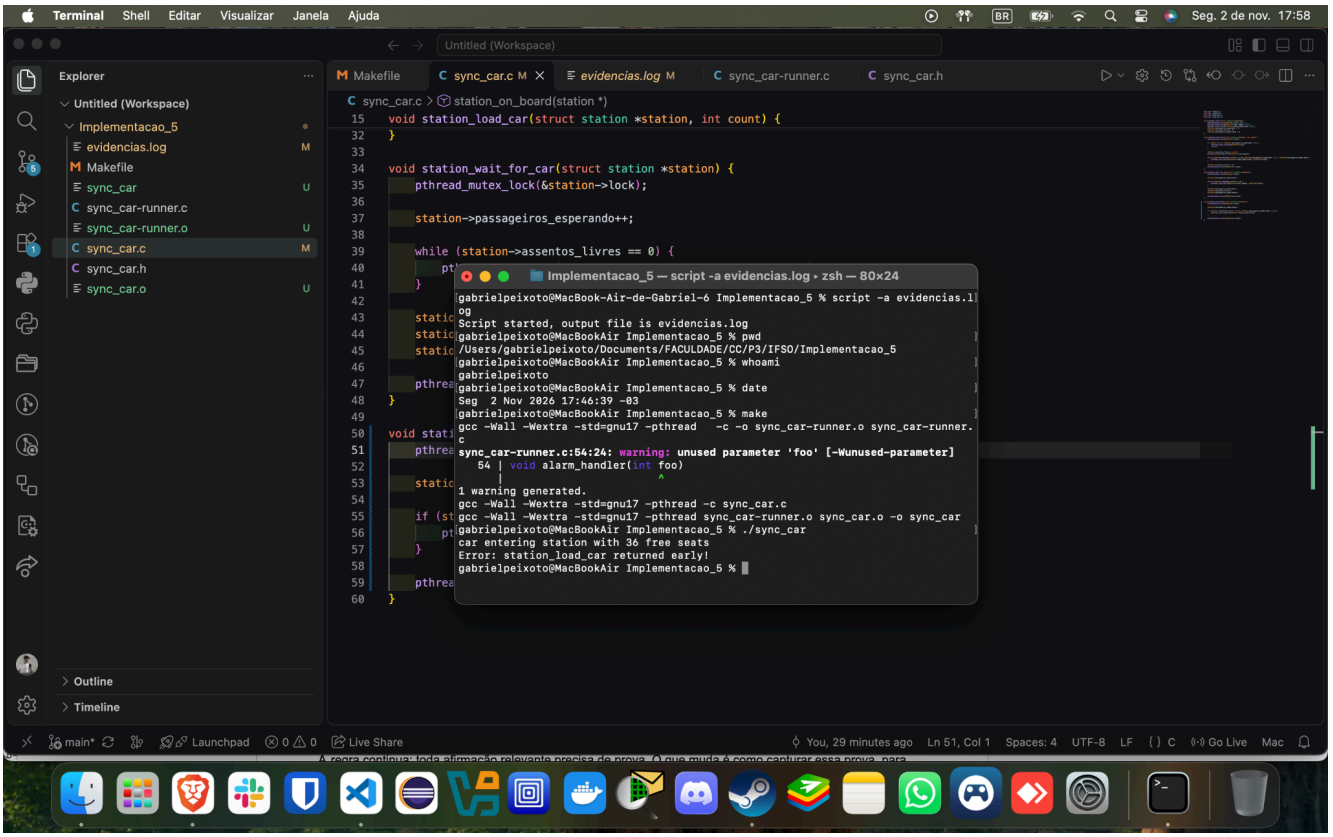
Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

• Erro principal: Falhas repetitivas na implementação do station_load_car

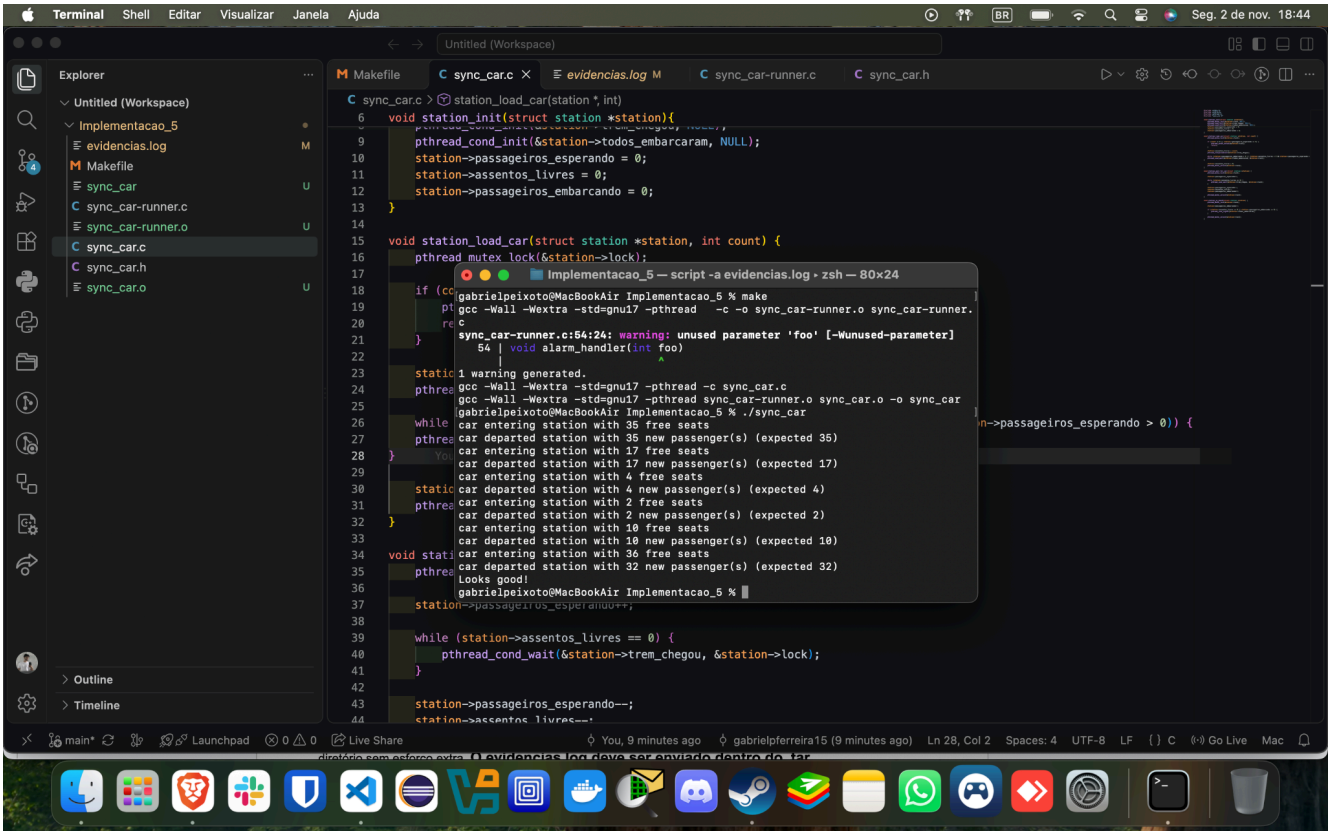




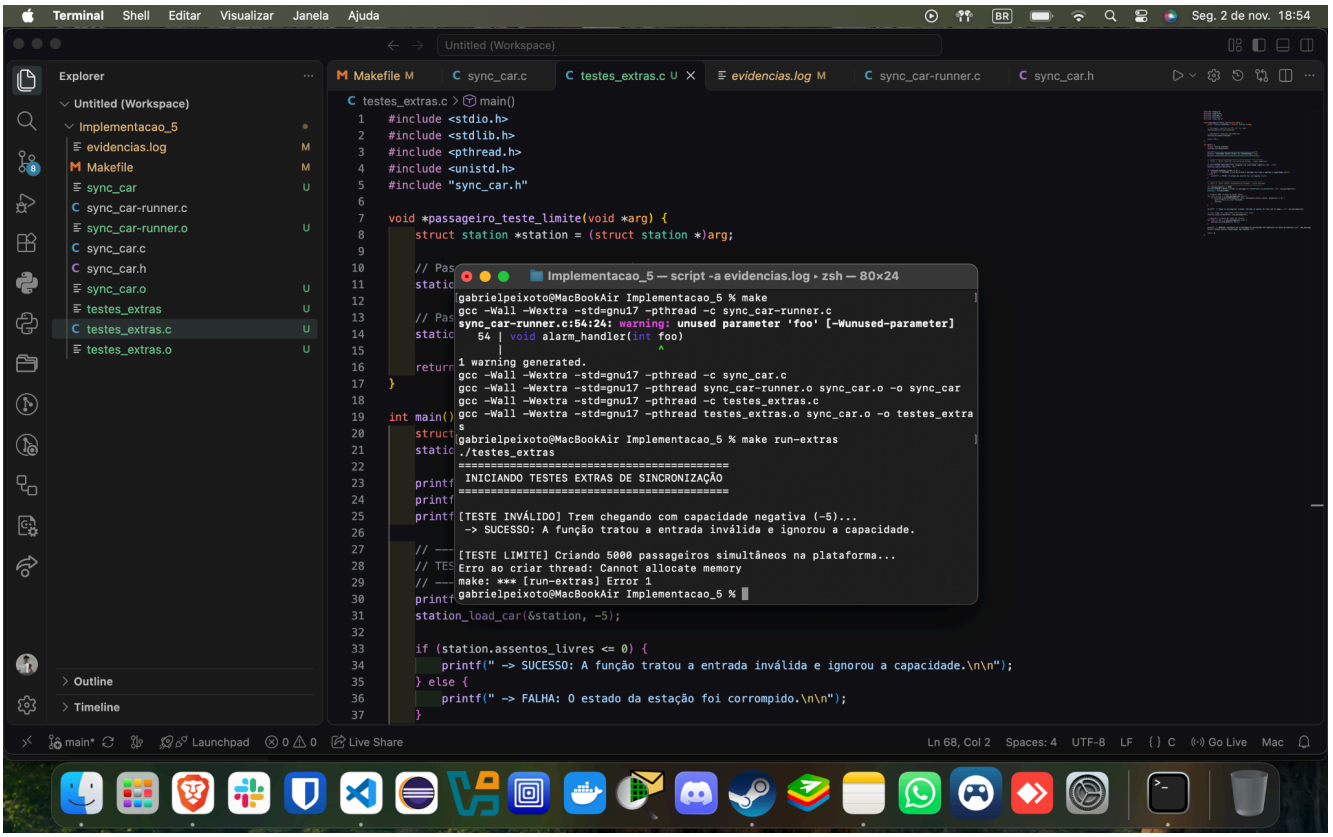


• Validação final:

Teste padrão (sync_car-runner):



Teste inválido + Teste limite:



7. Uso de IA

Onde usei IA (2–3 linhas):

A IA foi utilizada para entender melhor os conceitos da sincronização e como posso aplicar eles. Além disso, usei ela para tirar algumas dúvidas sobre o funcionamento e lógica de algumas funções.

Prompts principais (liste, sem história):

Me explique, no contexto de infraestrutura de software, o que é sincronização e seus conceitos básicos. Me mostre exemplos e me fale como eu posso aplicar tais conceitos para criar programas capazes de sincronizar threads de forma correta.

Me fale quais são as funções de pthreads e para que cada uma serve. Me explique detalhadamente aquelas que são úteis no contexto de sincronização e sistemas, falando sobre a sintaxe e lógica de funcionamento de cada uma.

Me explique melhor como funciona as funções de pthread_mutex_lock e pthread_mutex_unlock.

Gere para mim um teste inválido e um teste limite para que eu possa testar o código da implementação que fiz, que tem como escopo a tarefa do PDF anexado: 5_Syncronization.docx.pdf

O que validei manualmente e como:

Todas as decisões de arquitetura geral, funções utilizadas, organização do código e lógica geral do projeto foram feitas manualmente ao me basear em pesquisas/consultas feitas em documentações oficiais e em sites de discussão na internet como o Stack Overflow.

8. Reflexão Final

Reflexão:

A implementação deste sistema evidenciou a complexidade sutil da programação concorrente. A principal lição prática foi compreender que uma falha aparentemente inofensiva, como omitir um pthread_mutex_unlock em uma rota alternativa de um if, resulta no travamento sistêmico da aplicação. Por fim, vale ressaltar que trabalhar com variáveis de estado mostrou que a sincronização correta não se resume apenas a trancar recursos, mas sim a modelar com exatidão a relação de dependência temporal entre as threads, garantindo que uma ação só prossiga quando todo o contexto for favorável.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Eu focaria na otimização e investigação de limites do sistema operacional frente ao excesso de threads, pois durante a execução do meu teste limite para simular 5000 passageiros simultâneos, o terminal retornou a falha "Erro ao criar thread: Cannot allocate memory". Logo, dedicaria esse tempo extra para implementar uma arquitetura baseada em Thread Pool, reaproveitando threads em vez de criá-las de uma vez, ou para reconfigurar os limites de alocação de memória do meu ambiente, permitindo que a aplicação sustentasse simulações de carga ainda mais massivas sem estourar os recursos de hardware.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.